# Concevoir une application au service de la santé publique



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

  C'est un point essentiel pour moi car comme dirait Jane Goodall, anthropologue et ethnologue britannique :
  « Nous sommes ce que nous mangeons »

  Les variables sélectionnés sont :

  - Le nombre d'additifs
  - La présence d'ingrédients issus de l'huile de palme
  - La présence d'un label bio
  - Le nutriscore
  - Le taux de sucre (glucides)
  - Le taux de graisse (lipides)
  - Le taux de sel

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

  C'est un point essentiel pour moi car comme dirait Jane Goodall, anthropologue et ethnologue britannique :
  « Nous sommes ce que nous mangeons »

  Les variables sélectionnés sont donc au nombre de 28 :

  - 'code' , 'product_name' , 'additives_n', 'additives', 'additives_tags', 'additives_fr', 'ingredients_from_palm_oil_n', 'ingredients_from_palm_oil', 'ingredients_from_palm_oil_tags', 'ingredients_that_may_be_from_palm_oil_n', 'ingredients_that_may_be_from_palm_oil', 'ingredients_that_may_be_from_palm_oil_tags', 'fat_100g', 'saturated-fat_100g', 'monounsaturated-fat_100g', 'polyunsaturated-fat_100g', 'omega-3-fat_100g', 'omega-6-fat_100g', 'omega-9-fat_100g', 'trans-fat_100g', 'cholesterol_100g', 'sugars_100g', 'sucrose_100g', 'glucose_100g', 'salt_100g', 'sodium_100g', 'nutrition-score-fr_100g', 'nutrition-score-uk_100g', 'glycemic-index_100g'

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

## Définition du Nutri-Score :

« Il informe sur la qualité nutritionnelle simplifiée d'un aliment. Il est construit sur la base d'un score nutritionnel élaboré initialement par une équipe d'Oxford pour la Food Standard Agency (FSA) du Royaume-Uni. Il se base sur 5 couleurs associé à des lettres : le vert A étant la meilleure qualité et le rouge E le moins bon.

Ce score est basé sur les compositions nutritionnelles de 100g de produit tel que vendu et allie deux types de composantes :

- des facteurs nutritionnels à limiter (en cas de consommation excessive) : l'énergie (kJ/100g), les acides gras saturés (g/100g), les sucres simples (g/100g) et le sel (mg/100g)
- des facteurs nutritionnels à favoriser : les fibres (g/100g), les protéines (g/100g), les fruits et légumes, légumineuses et fruits à coque (g/100g)

Le score final dépendant de ces deux composantes permet d'attribuer une lettre et une couleur par soustraction de la composante négative par la composante positive.

Le nutri-score comporte des catégories et seuils spécifiques pour les matières grasses, les boissons et les fromages

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

## Définition du Nutri-Score

Nutri-score tient compte essentiellement du gras, sel, sucre, fibres, protéines et fruits-légumes-légumineuse-fruits à coque,

Pour simplifier le travail, notre Nutri-score dépendra essentiellement de trois facteurs : le gras, le sucre et le sel. Nous tenterons de déterminer les liens et corrélations entre ces variables.

Je souhaite aussi tenir compte en plus du Nutri-score (afin d'avoir une alimentation de qualité) :

- De huile de palme,
- Des additifs,
- D'un label bio défini dans le notebook

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

1ère étape : repérer les colonnes qui vont m'aider à construire mon application

```
df1 =  pd.DataFrame(columns = np.arange(162))

print(df_food.columns.tolist())
type(df_food.columns.tolist())
```

```
['code', 'url', 'creator', 'created_t', 'create
ic_name', 'quantity', 'packaging', 'packaging_t
'origins', 'origins_tags', 'manufacturing_place
s', 'emb_codes_tags', 'first_packaging_code_geo
_tags', 'countries_fr', 'ingredients_text', 'al
'no_nutriments', 'additives_n', 'additives', 'a
_palm_oil', 'ingredients_from_palm_oil_tags', '
l', 'ingredients_that_may_be_from_palm_oil_tags
'states', 'states_tags', 'states_fr', 'main_cat
gy-from-fat_100g', 'fat_100g', 'saturated-fat_1
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

1ère étape : repérer les colonnes qui vont m'aider à construire mon application

```
df_food1.isna().mean().sort_values(ascending=True)
```

```
Bio_score                                0.000000
code                                     0.000072
product_name                             0.055373
energy_100g                              0.185986
proteins_100g                            0.189699
salt_100g                                0.203453
sodium_100g                              0.203599
additives_n                              0.223938
ingredients_from_palm_oil_n              0.223938
ingredients_that_may_be_from_palm_oil_n  0.223938
sugars_100g                              0.236308
fat_100g                                 0.239675
saturated-fat_100g                       0.284370
nutrition-score-uk_100g                  0.310382
nutrition-score-fr_100g                  0.310382
fiber_100g                               0.373742
cholesterol_100g                         0.550802
trans-fat_100g                           0.553271
polyunsaturated-fat_100g                 0.928738
monounsaturated-fat_100g                 0.928850
fruits-vegetables-nuts_100g              0.990535
omega-3-fat_100g                         0.997378
omega-6-fat_100g                         0.999414
sucrose_100g                             0.999776
glucose_100g                             0.999919
omega-9-fat_100g                         0.999935
glycemic-index_100g                      1.000000
dtype: float64
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

La colonne Bio-score a été créée ! Méthode : diapo suivante

- Problème : très peu de données pour la colonne « fruits-légumes-fruits à coque ! », importante pour calculer le Nutri-score

```
df_food1.isna().mean().sort_values(ascending=True)
```

```
fruits-vegetables-nuts_100g                    0.990535
```

Donc sélection de mes colonnes définitives tient compte du pourcentage de données manquantes.

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

La colonne Bio-score a été créée !

**Méthode** :

```
df_food['Bio'] = df_food['labels'].str.contains('bio|Bio|AB')
```

```
df_food['Bio_score'] = 10 * df_food['Bio']
df_food['Bio_score']
```

```
df_food['Bio_score'] = np.where(df_food['Bio_score']==10, df_food['Bio_score'], 0 )
```

```
df_food['Bio_score']
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Ensuite, sélection des colonnes suivantes :

```
df_food2.isna().mean().sort_values(ascending=True)
```

```
Bio_score                                0.000000
code                                     0.000072
product_name                             0.055373
energy_100g                              0.185986
proteins_100g                            0.189699
salt_100g                                0.203453
sodium_100g                              0.203599
additives_n                              0.223938
ingredients_from_palm_oil_n              0.223938
ingredients_that_may_be_from_palm_oil_n  0.223938
sugars_100g                              0.236308
fat_100g                                 0.239675
nutrition-score-fr_100g                  0.310382
nutrition-score-uk_100g                  0.310382
fiber_100g                               0.373742
dtype: float64
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Traitement de mes colonnes :

Tenir compte des valeurs aberrantes :

```
df_food2['sodium_100g'] = np.where((df_food['sodium_100g']<=100) & (df_food['sodium_100g']>=0),
                                   df_food['sodium_100g'], np.NaN)
```

```
df_food2['saturated-fat_100g'] = np.where((df_food['saturated-fat_100g']<=100) &
                                          (df_food['saturated-fat_100g']>=0),
                                          df_food['saturated-fat_100g'], np.NaN)
```

```
df_food2['sugars_100g'] = np.where((df_food['sugars_100g']<=100) & (df_food['sugars_100g']>=0),
                                   df_food['sugars_100g'], np.NaN)
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Traitement de mes colonnes :

Tenir compte de la somme de mes trois composantes : si > 100, alors ajuster avec « la règle de trois » :

```
def proportion_correction(ingredient_mass, total_mass):
    if (total_mass <= 100):
        return ingredient_mass
    return (100 * ingredient_mass / total_mass)
    for ingredient_col in ['sodium_100g', 'saturated-fat_100g', 'sugars_100g']:
        df_food2[ingredient_col] = list(map(proportion_correction,
                                            df_food2[ingredient_col],
                                            df_food2['total_100g']))
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Parmi les colonnes du df_food2 sélectionnées, nous allons isoler les colonnes saturated-fat_100g, sodium_100g, sugars_100g (et nutrition-score-fr_100g) et écrire la fonction utilisée pour le calcul du Nutri-score qui convertit les trois premières colonnes en scores :

```
def score_sucre(x):
    if pd.isnull(x):
        return x
    if x <= 4.5:
        return 0
    if x <= 9:
        return 1
    if x <= 13.5:
        return 2
    if x <= 18:
        return 3
    if x <= 22.5:
        return 4
    if x <= 27:
        return 5
    if x <= 31:
        return 6
    if x <= 36:
        return 7
    if x <= 40:
        return 8
    if x <= 45:
        return 9
    return 10
```

```
def score_gras(x):

    if pd.isnull(x):
        return x
    if x <= 10 :
        return int(x)
    return 10
```

```
def score_sel(x):

    if pd.isnull(x):
        return x
    if x <= 0.9:
        return x//0.09
    return 10
```

```
df_food2['score-sugars2'] = df_food2['sugars_100g'].apply(score_sucre)
```

```
df_food2['score-gras2'] = df_food2['saturated-fat_100g'].apply(score_gras)
```

```
df_food2['score-sel2'] = df_food2['sodium_100g'].apply(score_sel)
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Nous allons maintenant traiter notre Dataframe df_food2 :

```
df_food2 = df_food2.dropna(subset=['code'])
```

```
df_food2 = df_food2.sort_values(by=['code','product_name'], na_position='last')
```

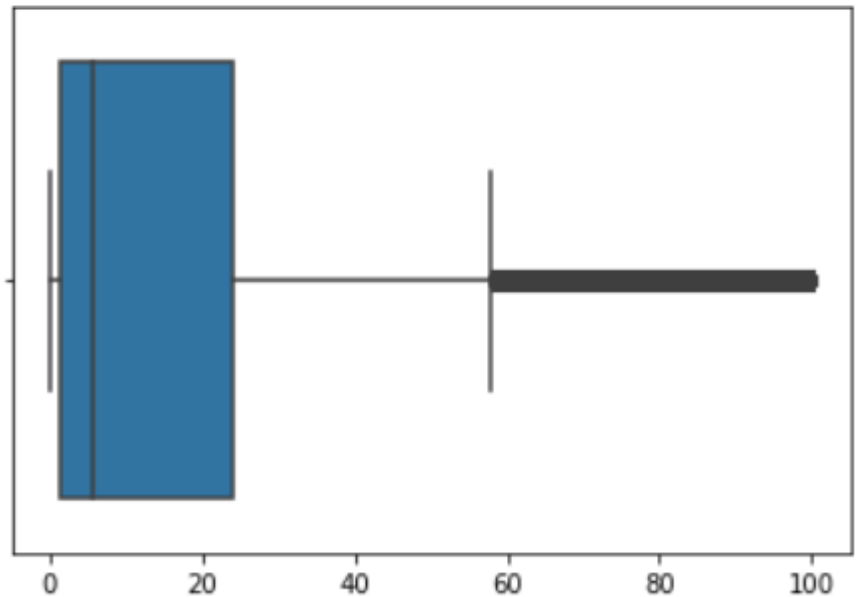
```
df_food2 = df_food2.drop_duplicates(subset=['code','product_name'], keep='first')
```

# 1. Traitement de jeu de données

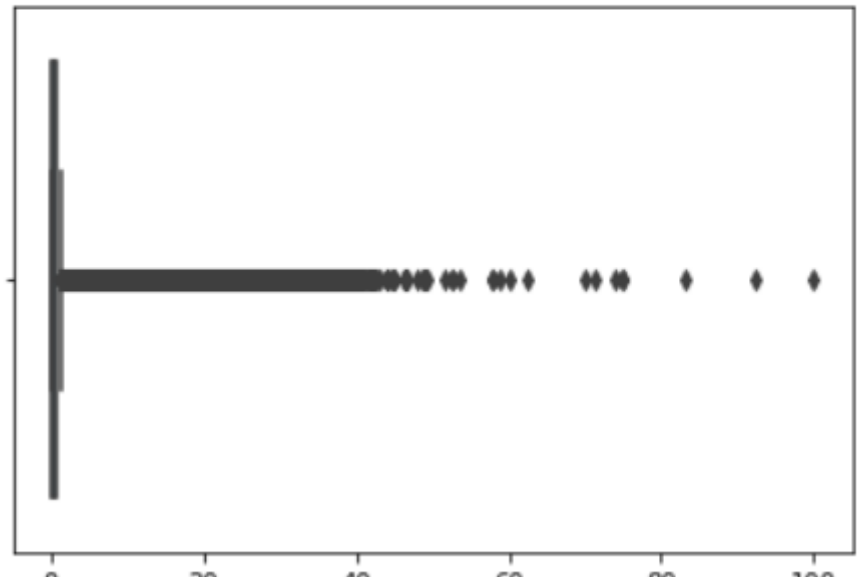
- Idée d'application : Trouver des produits sains pour mieux manger.

Au passage, regardons graphiquement ce que nous obtenons pour nos quatre colonnes, après traitement de valeurs aberrantes. (analyse univariée) :

```
sns.boxplot(x="sugars_100g",
            showfliers=True,
            orient='h',
            data=df_food2);
```




```
sns.boxplot(x="sodium_100g",
            showfliers=True,
            orient='h',
            data=df_food2);
```

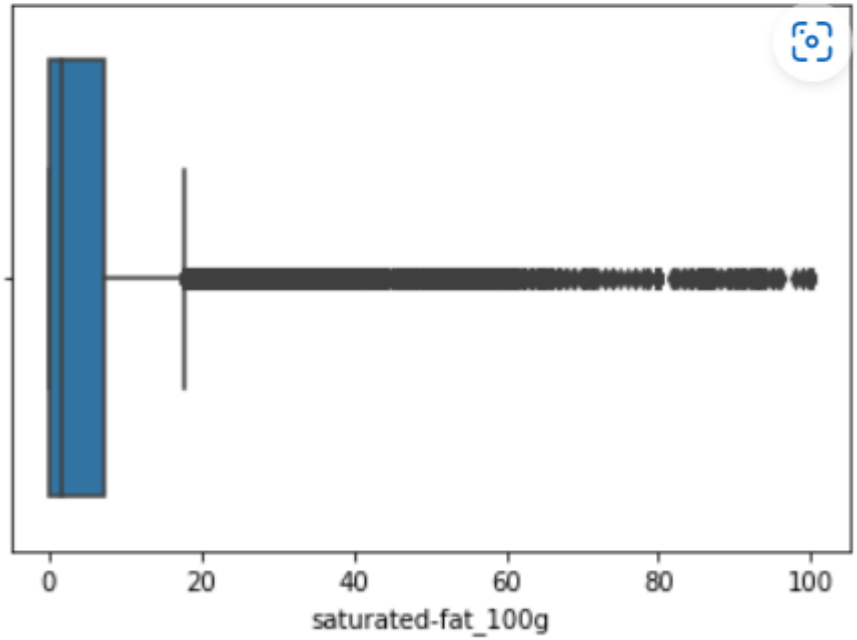


# 1. Traitement de jeu de données

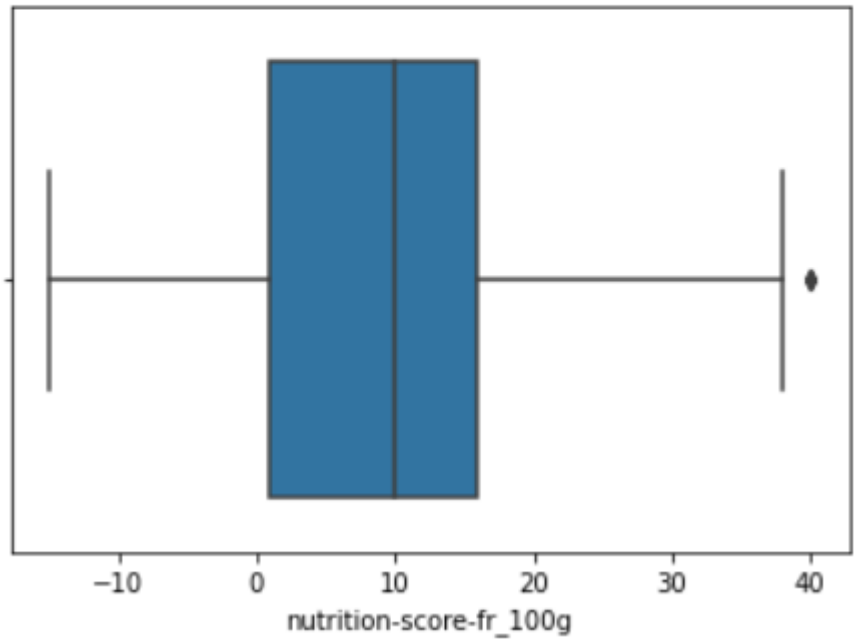
- Idée d'application : Trouver des produits sains pour mieux manger.

Au passage, regardons graphiquement ce que nous obtenons pour nos quatre colonnes (analyse univariée) :

```
sns.boxplot(x="saturated-fat_100g",
            showfliers=True,
            orient='h',
            data=df_food2);
```




```
sns.boxplot(x="nutrition-score-fr_100g",
            showfliers=True,
            orient='h',
            data=df_food2);
```



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Au passage, regardons graphiquement ce que nous obtenons pour nos quatre colonnes (ANOVA) :

```
#ANOVA
df_melt = pd.melt(df_food2.reset_index(), id_vars=['index'],
                  value_vars=['sodium_100g', 'saturated-fat_100g',
                              'sugars_100g', 'nutrition-score-fr_100g'])

df_melt.columns = ['index', 'treatments', 'value']
```

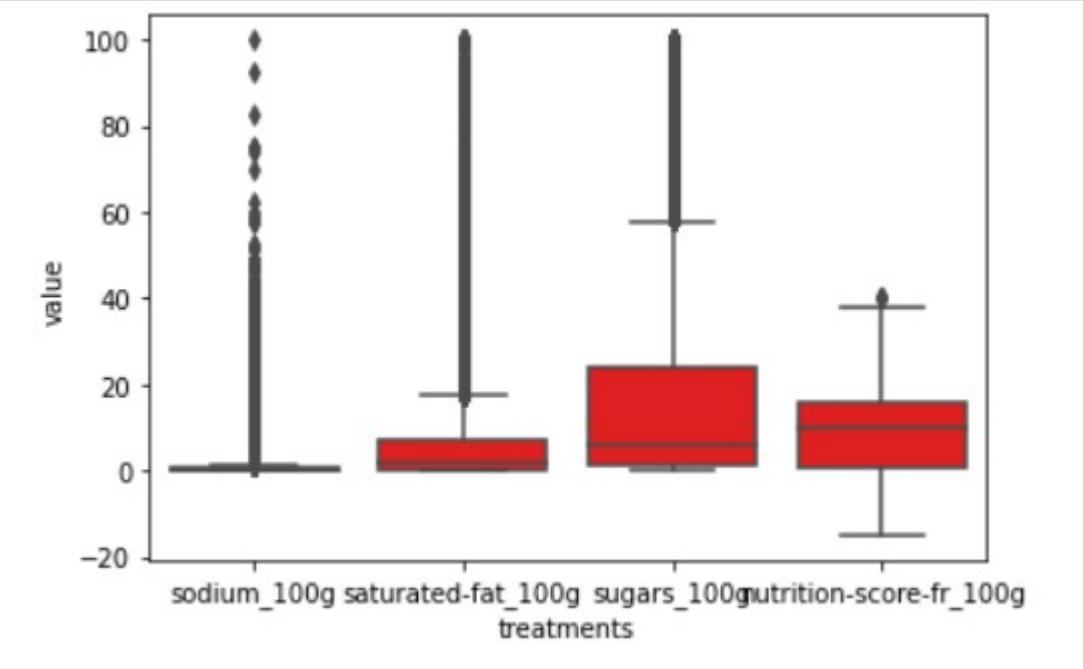
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
ax = sns.boxplot(x='treatments', y='value', data=df_melt, color='r')
plt.show()
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Au passage, regardons graphiquement ce que nous obtenons pour nos quatre colonnes (ANOVA) :



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Au passage, regardons graphiquement ce que nous obtenons pour nos quatre colonnes (ANOVA) :

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

mod = ols('value ~ treatments', data=df_melt).fit()

aov_table = sm.stats.anova_lm(mod, typ=2)
print (aov_table)
```

```
                sum_sq        df             F  PR(>F)
treatments  3.145564e+07       3.0  69514.693758     0.0
Residual    1.434644e+08  951138.0           NaN     NaN
```

```
esq_sm = aov_table['sum_sq'][0]/(aov_table['sum_sq'][0]+
                                 aov_table['sum_sq'][1])
esq_sm
```

```
0.17982866566847722
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Nous allons maintenant traiter notre Dataframe df_food2 :

- Nous pourrions remplacer les valeurs manquantes dans notre Dataframe par :

```
df_food2.loc[df_food2['score_sugars2'].isna(), 'score_sugars2'] = 0
```

Ou bien :

```
df_food2.loc[df_food2['score_sel2'].isna(), 'score_sel2'] = df_food2['score_sel2'].median()
```

Ou encore :

```
df_food2.loc[df_food2['score_gras2'].isna(), 'score_gras2'] = df_food2['score_gras2'].mean()
```

Mais aucune de ces méthodes n'est adapté malheureusement (NaN ne veut pas forcément dire 0, il y a ces deux « valeurs » dans les colonnes en question). Aussi calculer une moyenne du gras ou la médiane du sel fausserait nos colonnes (car chaque individu, ici produit a une composition qui lui est singulière et qui va déterminer le score)

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Donc les lignes avec des NaN vont être dans un premier temps omis pour réaliser une ACP (à titre illustratif) afin de déterminer les corrélations entre nos différentes variables qui nous intéressent.

```
# réalisation de l'ACP
import sklearn
from sklearn import decomposition
from sklearn import preprocessing
from sklearn import datasets

from mpl_toolkits.mplot3d import Axes3D

from functions import *
```

```
n_comp2 = 6

data_pca2 = df_food2[['score-sugars2','score-sel2','score-gras2',
                      'Bio_score', 'additives_n',
                      'ingredients_from_palm_oil_n']]
```

```
# préparation des données pour l'ACP
data_pca2 = data_pca2.dropna()
X2 = data_pca2.values
names2 = df_food2.index
features2 = data_pca2.columns
```

```
# Centrage et Réduction
std_scale = preprocessing.StandardScaler().fit(X2)
X2_scaled = std_scale.transform(X2)
```

```
# Calcul des composantes principales
pca2 = decomposition.PCA(n_components=n_comp2)
pca2.fit(X2_scaled)
```

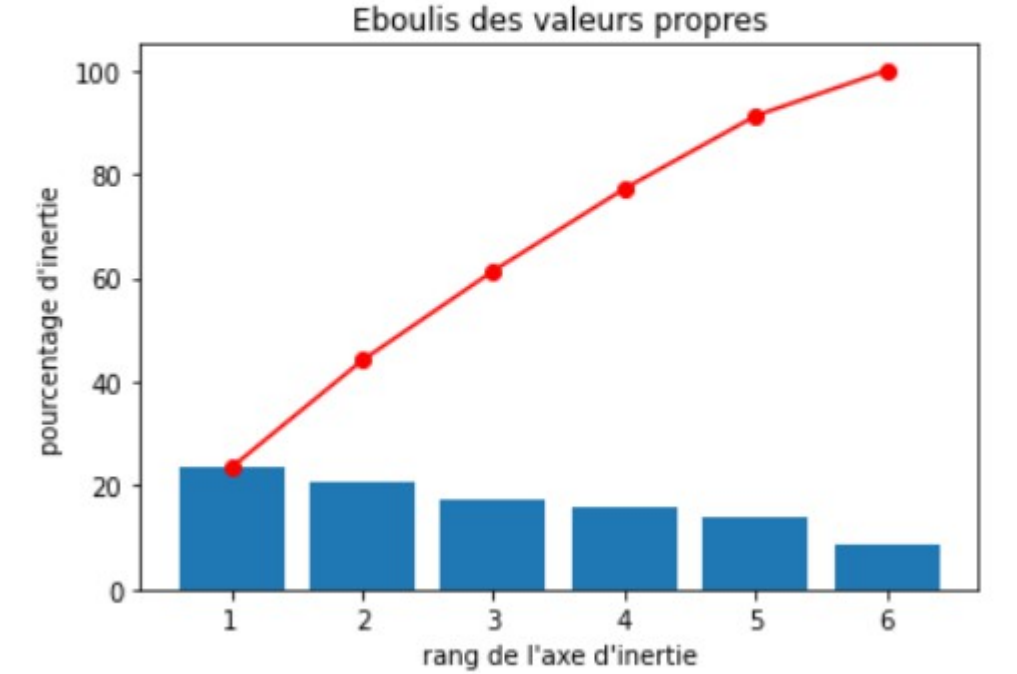
```
PCA(n_components=6)
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Réalisation de l' ACP afin de déterminer les corrélations entre nos différentes variables.

```
# Eboulis des valeurs propres
display_scree_plot(pca2)
```



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

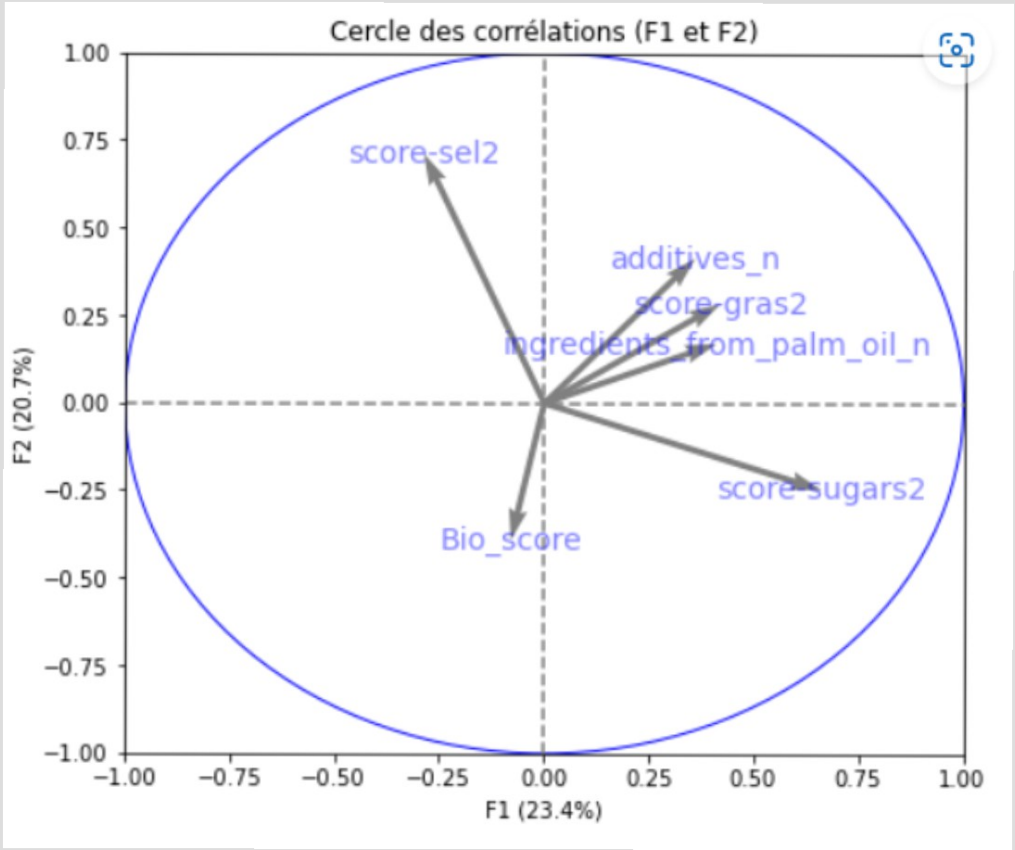
Réalisation de l' ACP afin de déterminer les corrélations entre nos différentes variables.

```
# Cercle des corrélations
pcs2 = pca2.components_
display_circles(pcs2, n_comp2, pca2, [(0,1),(2,3),(4,5)],
                labels = np.array(features2))
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Réalisation de l' ACP afin de déterminer les corrélations entre nos différentes variables.



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

  Maintenant, réalisation de l' ACP en intégrant le nutri-score afin de déterminer les corrélations entre les principales variables composant ce dernier et ce dernier.

```
n_comp = 4

data_pca = df_food2[['score-sugars2','score-sel2','score-gras2', 'nutrition-score-fr_100g']]
```

```
# préparation des données pour l'ACP
data_pca = data_pca.dropna()
X = data_pca.values
names = data_pca['nutrition-score-fr_100g'] # ou data.index pour avoir les intitulés
features = data_pca.columns
```

```
# Centrage et Réduction
std_scale = preprocessing.StandardScaler().fit(X)
X_scaled = std_scale.transform(X)
```

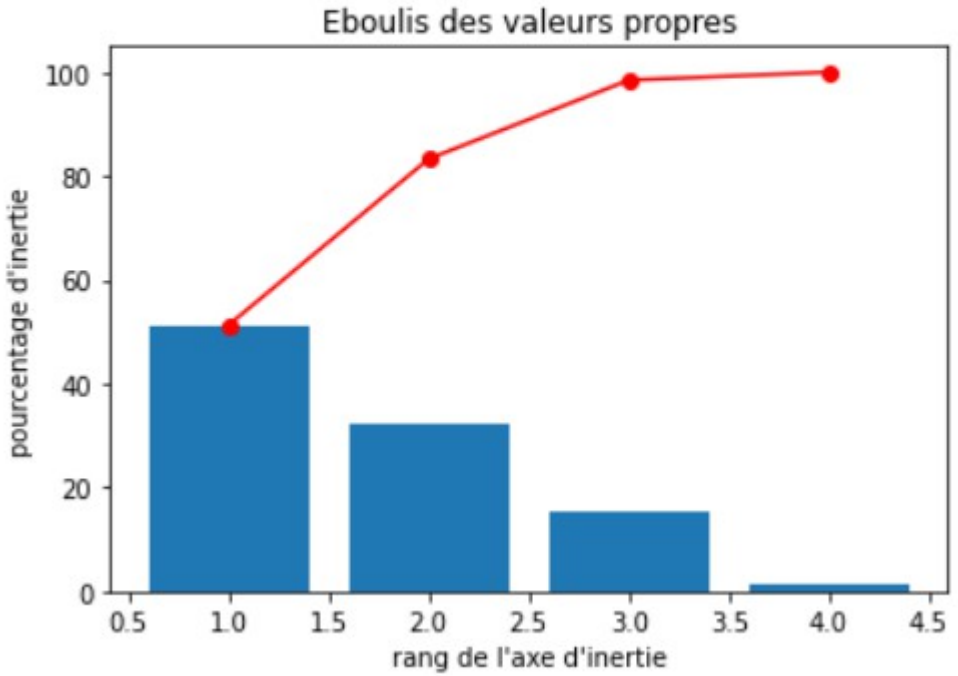
```
# Calcul des composantes principales
pca = decomposition.PCA(n_components=n_comp)
pca.fit(X_scaled)
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Maintenant, réalisation de l' ACP en intégrant le nutri-score afin de déterminer les corrélations entre les principales variables composant ce dernier et ce dernier.

```
# Eboulis des valeurs propres
display_scree_plot(pca)
```



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

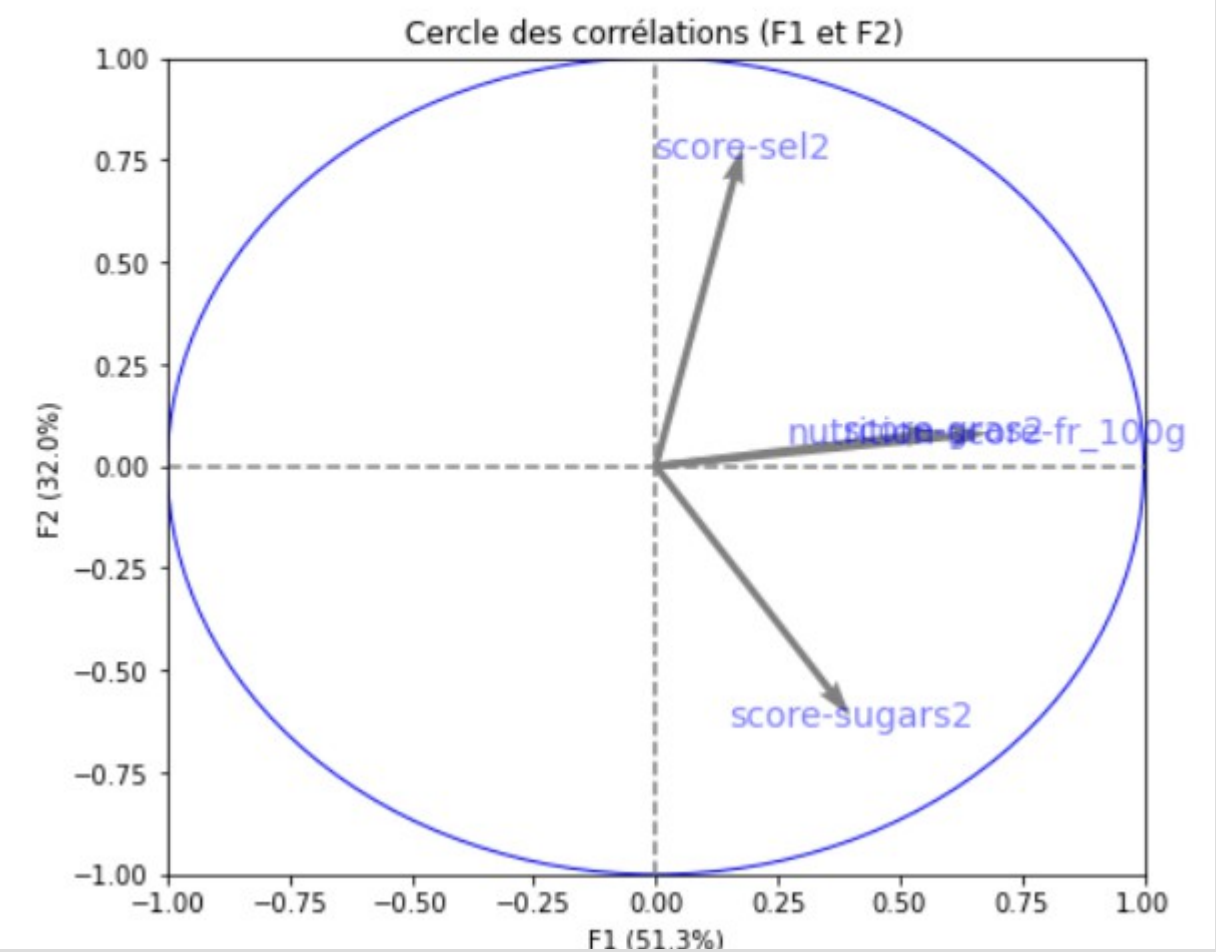
Maintenant, réalisation de l' ACP en intégrant le nutri-score afin de déterminer les corrélations entre les principales variables composant ce dernier et ce dernier.

```
# Cercle des corrélations
pcs = pca.components_
display_circles(pcs, n_comp, pca, [(0,1),(2,3),(4,5)],
                labels = np.array(features))
```

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Maintenant, réalisation de l' ACP en intégrant le nutri-score afin de déterminer les corrélations entre les principales variables composant ce dernier et ce dernier.



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Ces corrélations correspondent à ce que nous pouvons lire ci-dessous :

```
correlation_matrix = df_food2[['nutrition-score-fr_100g','score-sugars2',
                               'score-gras2','score-sel2','Bio_score','additives_n',
                               'ingredients_from_palm_oil_n']].corr()
correlation_matrix["nutrition-score-fr_100g"]
```

```
nutrition-score-fr_100g        1.000000
score-sugars2                  0.517174
score-gras2                    0.759277
score-sel2                     0.352581
Bio_score                     -0.076211
additives_n                    0.167214
ingredients_from_palm_oil_n    0.117685
Name: nutrition-score-fr_100g, dtype: float64
```

- Nous voyons une corrélation assez importante entre le score-gras2 et le nutriscore, nous ferons donc le choix de laisser la variable nutrition-score-fr_100g de côté pour le calcul final de notre nouveau score (l'alternative au nutri-score rappelons-le)

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

  Ces corrélations correspondent à ce que nous pouvons voir ci-dessous :

```
import seaborn as sns

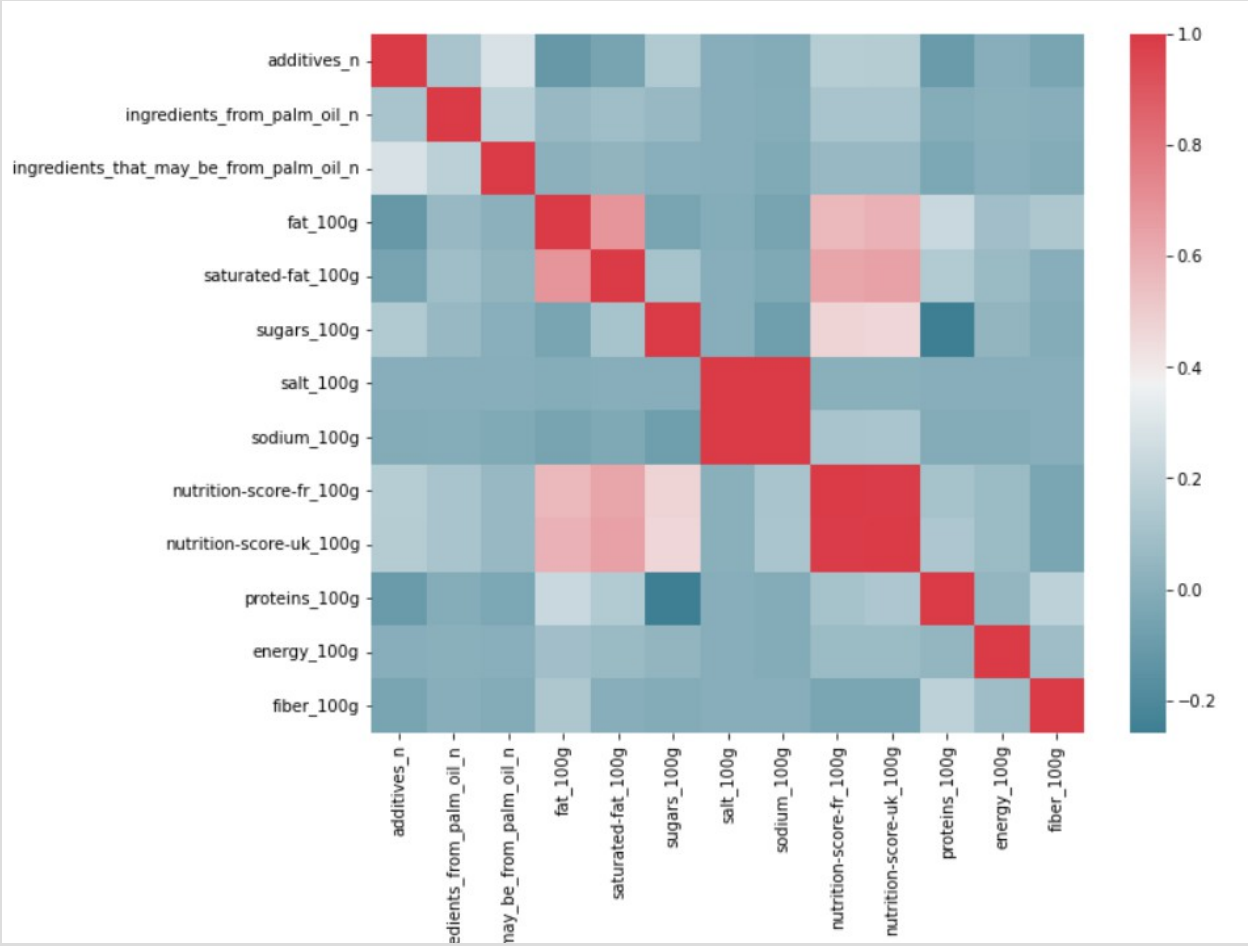
f, ax = plt.subplots(figsize=(10, 8))
corr = df_food2.corr()
sns.heatmap(corr, mask=np.zeros_like(corr, dtype=np.bool),
            cmap=sns.diverging_palette(220, 10, as_cmap=True),
            square=True, ax=ax)
```

- Nous voyons une corrélation assez importante entre le score-gras2 et le nutriscore, nous ferons donc le choix de laisser la variable nutrition-score-fr_100g de côté pour le calcul final de notre nouveau score (l'alternative au nutri-score rappelons-le)

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Ces corrélations correspondent à ce que nous pouvons voir ci-dessous :



# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

Maintenant regardons les corrélations entre nos variables utilisées lors de la 1ère ACP présentée (celle avec les labels bio, huile de palme et additifs, sans le nutri-score) :

```
correlation_matrix = df_food2[['score-sugars2',
                               'score-gras2','score-sel2','Bio_score','additives_n',
                               'ingredients_from_palm_oil_n']].corr()
correlation_matrix
```

| | score-sucres2 | score-gras2 | score-sel2 | Bio_score | additives_n | ingredients_from_palm_oil_n |
|---|---|---|---|---|---|---|
| **score-sucres2** | 1.000000 | 0.203010 | -0.285890 | -0.029471 | 0.154922 | 0.078133 |
| **score-gras2** | 0.203010 | 1.000000 | 0.103913 | -0.028645 | 0.009797 | 0.129024 |
| **score-sel2** | -0.285890 | 0.103913 | 1.000000 | -0.078922 | 0.072002 | -0.018220 |
| **Bio_score** | -0.029471 | -0.028645 | -0.078922 | 1.000000 | -0.107236 | 0.019731 |
| **additives_n** | 0.154922 | 0.009797 | 0.072002 | -0.107236 | 1.000000 | 0.119898 |
| **ingredients_from_palm_oil_n** | 0.078133 | 0.129024 | -0.018220 | 0.019731 | 0.119898 | 1.000000 |

- Nous voyons ici qu'il y a très peu de corrélations entre les différentes variables, nous garderons donc ces dernières (à l'exception du score-gras2 et avec le nutri-score à la place) afin de calculer notre nouveau score que nous présenterons après avoir complété les valeurs manquantes notre jeu de données.

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

  En effet, pour terminer cette première partie, nous allons imputer les 'NaN' de notre df_food2 par la méthode KNN.

  En effet, cette imputation m'a parue la plus pertinente car il était plus « évident » pour moi de parler, de comparer avec les articles voisins, similaires. Ca nous permettra aussi et surtout à la fin de notre travail de tester la corrélation (ou non) entre notre nouveau score appelé Score_santé et le nutri-score

# 1. Traitement de jeu de données

- Idée d'application : Trouver des produits sains pour mieux manger.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.impute import KNNImputer
```

```
model = KNeighborsClassifier()
```

```
X_cols = ['score-sel2','score-sugars2','score-gras2','additives_n',
                'ingredients_from_palm_oil_n','Bio_score'
                 ]
```

```
X = df_food2[X_cols + ["nutrition-score-fr_100g"]]
```

```
df_food2[X_cols] = df_food2[X_cols].fillna(0)
```

```
imputer = KNNImputer(n_neighbors=3, weights="uniform", copy=False)
imputer.fit_transform(X)
```

# 2. Mise en place du scoring

- Idée d'application : Trouver des produits sains pour mieux manger.

```
#création de scores
X['SCORE-SUCRE'] = X['score-sugars2'] / X['score-sugars2'].max()
X['SCORE-SEL'] = X['score-sel2'] / X['score-sel2'].max()
X['SCORE-GRAS'] = X['score-gras2'] / X['score-gras2'].max()
X['SCORE-BIO'] = X['Bio_score'] / X['Bio_score'].max()
X['SCORE-ADDITIFS'] = X['additives_n'] / X['additives_n'].max()
X['SCORE-HUILE-PALME'] = X['ingredients_from_palm_oil_n'] /
                         X['ingredients_from_palm_oil_n'].max()
```

```
X['SCORE_SYNT'] = (X['SCORE-SUCRE'] + X['SCORE-SEL'] + X['SCORE-GRAS'] -
                   X['SCORE-BIO'] + X['SCORE-ADDITIFS'] + X['SCORE-HUILE-PALME'] ) / 6
X
```

```
X_definitif =  pd.DataFrame(index = [df_food2['code'] , df_food2['product_name'],
                                     X['SCORE-SUCRE'] , X['SCORE-SEL'] ,
                                     X['SCORE-GRAS'] , X['SCORE-BIO'] ,
    X['SCORE-ADDITIFS'] , X['SCORE-HUILE-PALME'] , X['SCORE_SYNT']])
```

# 2. Mise en place du scoring

- Idée d'application : Trouver des produits sains pour mieux manger.

Mise en place de notre nouveau score : Score_santé

```
def Score_sante (x):
    if pd.isnull(x):
        return x
    if x <= 1/6:
        return 1
    if x <= 2/6:
        return 2
    if x <= 3/6:
        return 3
    if x <= 4/6:
        return 4
    if x<= 5/6 :
        return 5
    if x<= 6/6:
        return 6
```

```
X['score_SANTE'] = X['SCORE_SYNT'].apply(Score_sante)
```

# 2. Mise en place du scoring

- Idée d'application : Trouver des produits sains pour mieux manger.

  Test corrélation entre « nutrition-score-fr_100g » et « Score-santé » :

  - Si faible corrélation, alors bonne alternative
  - Si forte corrélation, alors nutri-score suffisant

```
from scipy.stats import pearsonr
```

```
corr, _ = pearsonr(X['nutrition-score-fr_100g'], X['score_SANTE'])
print('Pearsons correlation: %.3f' % corr)
```